

Техническое задание: WindSpotBot

1. Общее описание

****Название****: WindSpotBot

****Цель****: Telegram-бот для кайтсерферов с функциями чек-ина, планирования, просмотра спотов, прогноза погоды, уведомлений, чатов с хэштегами, оценки спотов и админ-статистики. Подготовка к будущему веб-приложению.

****Платформа****: Telegram, библиотека `aiogram` 3.x.

****Язык****: Python 3.10+.

****Целевая аудитория****: Кайтсерферы, администраторы спотов.

****Сообщество****: `@WindSpotRU` для обсуждений и новостей.

****Особенности****:

- Морской, кайтерский, сёрфовый тон с юмором («лови ветер», «йо, бро»).
- Тематические эмодзи: 🌊, 🌤️, 🏄, 🌬️, 🌈, ☀️.
- Изолированные сервисы и репозитории для масштабируемости.
- Конфигурация через `.env` с валидацией `pydantic`.
- Тестирование (покрытие >80%) и документация.

2. Функциональные требования

2.1. Основные команды и сценарии

1. ****Команда `/start`****:

- Регистрирует пользователя (ID, имя, username, часовой пояс по геолокации).
- Показывает приветствие:

```
🌊 Йо, {name}! Добро пожаловать на борт WindSpotBot! 🏄
Лови ветер, находи споты и тусуйся с кайтерами! 🌬️
💬 Заходи в @WindSpotRU, там волна новостей и движухи!
```

- Динамическое меню (например, «Покинуть спот» для чек-ина типа 1).
- Логирование: `INFO: Пользователь user\_id=123 вызвал /start`.

2. ****Чек-ин****:

- ****Тип 1 («На месте», 🟢)****: Время пребывания 1-6 часов, автодеактивация.
- ****Тип 2 («Прибуду», 🕒)****: Время до прибытия 1-3 часа,

автодеактивация/удаление.

- ****Тип 3 («Планирование», 📅)****: Дата (до 7 дней), время, спот.
- Ограничение: Один активный чек-ин (типы 1 или 2).
- Использует кэшированную геолокацию (TTL=1 час).
- Показывает N ближайших спотов (`NEARBY\_SPOTS\_LIMIT`).
- Поддерживает создание нового спота.
- Уведомляет админов и пользователей избранных спотов.
- Пример:

```
🟢 Йо, ты на Спот 1! Лови вайб! 🏄
Сколько тусить будешь?
[1 час] [2 часа] [3 часа] [4 часа] [5 часов] [6 часов]
```

3. ****Планирование посещений****:

- Чек-ин типа 3.
- Данные: спот, дата, время.
- Отображается на страничке спота (N записей, `PLANNED\_VISITS\_LIMIT`):

```
📅 Планы на волну:
1. 20.04.2025 14:00, <a href="tg://user?id=123">@ivan123</a>
2. 21.04.2025 10:00, <a href="tg://user?id=456">@anna</a>
```

4. ****Кто на спотах****:

- Список активных спотов (N, в `.env`) с погодой и пользователями (без кэша):

```
🌬️ Ветер гонит волну! Проверь, кто тусит:
🏄 Спот 1 (1.2 км)
🌊 Ветер: 5 м/с (порывы 7 м/с), СЗ
```

 На месте: @ivan123 (ещё 2 часа)
Прибудут: @anna (через час)
 Планы: @petr (20.04.2025, 14:00)

5. **Ближайшие споты**, **Покинуть спот / Подтвердить прибытие**, **Профиль**, **Прогноз погоды**, **Избранные споты**:
- С морским тоном и эмодзи, как в предыдущих версиях ТЗ.

6. **Сообщения/чаты спотов**:
- Хэштег-чат (основной) в одном Telegram-чате (`CHAT_ID` в `.env``).
- Авто-вставка хэштегов (например, `#Спот1``).
- Фильтрация через `aiogram` (Message.text.contains("#Спот1")`).
- Пагинация (10 сообщений).
- Команда /filter #Спот1` для просмотра сообщений.
- Запасные варианты: группы (spot_chats`) или база (spot_messages`), с лёгкой заменой через ChatService`.
- Пример:`

 Новое сообщение в #Спот1 от @ivan123: Ветер огонь! 🌪️

7. **Страничка спота**:
- Данные: название, геолокация, описание, погода, рейтинг, статистика, планы.
- Кнопки: чек-ин, планирование, чат, избранное, оценка.
- Пример:

 **Спот 1**
 1.2 км от тебя
 Ветер: 5 м/с (порывы 7 м/с), СЗ
 Вода: 18°C
 Рейтинг: 4.5 (10 оценок)
 На месте: @ivan123
[Чек-ин] [Планировать] [Чат] [★ Оценить]

8. **Оценка спотов**:
- Оценка (1-5) через страничку спота.
- Уведомление после завершения чек-ина типа 1 (кнопка «Покинуть» или автодеактивация), задержка 5 минут:

 Как тебе Спот 1? Дай вайб от 1 до 5! 🌪️
[Оценить Спот 1]

- Одна оценка на пользователя, с возможностью изменения.
- Проверка в `spot_ratings`.`

9. **Команда `/help``**:
- Описание функций с морским тоном:

 **Лови волну помощи, бро!**
 `/start` — врывайся в движ!
 Отметиться — лови спот!
 Кто на спотах — чекни тусу
 /weather` — ветер в паруса
 Обсуждай вайб в @WindSpotRU! 🌪️`

10. **Админ-функции**:
- `/admin_notifications` : управление уведомлениями.
- /broadcast` : рассылка.
- /stats` (низкий приоритет): пользователи, споты, чек-ины, топ-5 спотов (кэш, TTL=10 минут).`

11. ****Геймификация**** (средний приоритет):

- Счётчик чек-инов:

 Бро, ты поймал 25 спотов! 

- Бейджи: «Новичок» (1), «Ветер в волосах» (10), «Мастер спотов» (50).
- Лидерборд: топ-5.

12. ****Социальные функции**** (средний приоритет):

- Поделиться чек-ином:

 Я на Спот 1! Ветер 5 м/с! #Спот1 
[Поделиться в чате]

- Пригласить друзей.

13. ****Умные рекомендации и частота уведомлений**** (низкий приоритет).

2.2. Уведомления

- Персонализированные, с морским тоном:

 Йо, {name}! На Спот 1 новый чек-ин от @anna! Лови волну? 

- События через `aioevents` (например, `CheckinCreatedEvent`).

2.3. Планировщик задач

- `SchedulerService` для:
 - Проверки чек-инов (`CHECKIN\_CHECK\_INTERVAL\_MINUTES`).
 - Обновления погоды (`WEATHER\_CHECK\_INTERVAL\_MINUTES`).
 - Удаления сообщений по TTL (`MESSAGE\_TTL\_DAYS`, если база).
- Реализация через `APScheduler`.

2.4. База данных

- ****Тип****: SQLite (`data/database.db`), миграции через `alembic` (универсально для PostgreSQL).

- ****Таблицы****:

- `users`: ID, имя, username, часовой пояс.
- `spots`: ID, название, координаты, описание.
- `checkins`: ID, user_id, spot_id, тип, время, статус.
- `favorite\_spots`: user_id, spot_id.
- `notifications\_settings`: user_id, настройки.
- `admin\_settings`: admin_id, настройки.
- `spot\_ratings`: user_id, spot_id, оценка.
- `achievements`: user_id, название, дата.

- ****Индексы****:

```
```sql
CREATE INDEX idx_checkins_spot_id ON checkins (spot_id, active_until);
CREATE INDEX idx_planned_visits_spot_id ON checkins (spot_id, planned_at);
```
```

3. UX/UI

3.1. Принципы

- ****Тон****: Морской, кайтерский, сёрфовый, с юмором («лови ветер», «йо, бро»).

- ****Эмодзи****:

- Основные:  ,  ,  ,  ,  ,  .
- Дополнительные:  ,  ,  ,  ,  .

- ****Клавиатуры****:

- Inline: максимум 3 кнопки в ряду.
- Reply: геолокация, текст.
- « Назад» на промежуточных шагах, « Меню» после подтверждения.
- ****Прогресс****: «Шаг 1/3» в многошаговых сценариях.

- ****Ошибки****:

 Упс, волна унесла твою геолокацию!  Отправь ещё раз, бро!

...

3.2. Примеры сообщений

- **Приветствие**:

...

👉 Йо, {name}! Добро пожаловать на борт WindSpotBot! 🚤
Лови ветер, находи споты и тусуйся с кайтерами! 🌊
💬 Заходи в @WindSpotRU, там волна новостей и движухи!

- **Чек-ин**:

...

✅ Йо, ты на Спот 1! Лови вайб! 🚤
Сколько тусить будешь?
[1 час] [2 часа] [3 часа] [4 часа] [5 часов] [6 часов]

- **Оценка**:

...

👉 Как тебе Спот 1? Дай вайб от 1 до 5! 👉
[Оценить Спот 1]

4. Нефункциональные требования

1. **Код**:

- PEP 8, проверка через `flake8`, `black`.
- Type hints, докстринги на русском.
- Комментарии для сложной логики.

2. **Изоляция**:

- Сервисы (`services/\*`) не зависят от хендлеров (`handlers/\*`).
- Репозитории (`repositories/\*`) для БД.
- Конфигурация через `pydantic` в `config.py`.

3. **Кэширование**:

- Вариант 1:
 - Кэш: список спотов, расстояние (TTL=1 час), погода (TTL=15 минут), статистика (TTL=10 минут).
 - Без кэша: активные пользователи (чек-ины, планы).
 - Сброс кэша при новых координатах:

```
```python
await SpotService().invalidate_spot_cache(lat, lon)
```
```

- Оптимизация: индексы в БД.

4. **Масштабируемость**:

- CQRS: `CheckinCommandHandler`, `SpotQueryHandler`.
- События через `aioevents`.
- Миграции через `alembic`.
- API-готовность для веб-приложения.

5. **Логирование**:

- Через `logging.config`, ротация логов (10MB, 5 файлов).
- Формат: `%(asctime)s - %(name)s - %(levelname)s - %(message)s`.

6. **Тестирование**:

- Юнит-тесты (`pytest`, `pytest-asyncio`).
- Моки для API (`unittest.mock`).
- Интеграционные тесты для сценариев.
- Покрытие >80% (`pytest-cov`).
- A/B-тестирование клавиатур (3 vs 2 кнопки).

7. **Документация**:

- `README.md`: установка, запуск, зависимости.
- `docs/`: архитектура, UX (`ux.md`), API (`api.md`).
- `CHANGELOG.md` с Conventional Commits.

5. Технологии

- Python 3.10+.
- `aiogram` 3.19.0.
- `aiosqlite`, `alembic`.
- `aiocache`, `aiohttp` (Open-Meteo), `timezonefinder`.

- `pydantic`, `python-dotenv`.
- `pytest`, `pytest-asyncio`, `flake8`, `black`.
- `APScheduler`, `aioevents`.

6. Ограничения

- Только Telegram.
- SQLite на старте, с переходом на PostgreSQL.
- Русский язык (многоязычность позже).
- Погода обновляется каждые 15 минут.

7. Приоритеты

- ****Высокий****: Чек-ины, «Кто на спотах», уведомления, страничка спота, оценка, чаты.
- ****Средний****: Планирование, погода, профиль, геймификация, социальные функции.
- ****Низкий****: `/stats`, рекомендации, настройка уведомлений.

8. Конфигурация

```
```env
BOT_TOKEN=your_token
CHAT_ID=chat_id
ADMINS=123,456
CHECKIN_CHECK_INTERVAL_MINUTES=5
WEATHER_CHECK_INTERVAL_MINUTES=15
MESSAGE_TTL_DAYS=7
NEARBY_SPOTS_LIMIT=5
PLANNED_VISITS_LIMIT=5
STATS_CACHE_TTL_SECONDS=600
```
```

9. План на веб-приложение

- Универсальные сервисы и репозитории.
- Документация API в `docs/api.md`.
- Подготовка к Redis для кэширования.